Hochschule Karlsruhe · Wintersemester 2026/2027

Industrial AI

Datenvorverarbeitung, Feature-Erzeugung und Transformation

Session 2 im Gesamtprozess: Schritte 02–04 · Vorbereitung der Modellierung.

02



Synthetisches Lehrbeispiel

Session 2

180 Minuten · EDA, Features & Teamarbeit

Dr.-Ing. Yannic Wolf

CTO & Geschäftsführer
Industrial AI GmbH

Ablauf der Session

4 Teams · zwei zusammenhängende Arbeitsblöcke mit 45 und 55 Minuten

0–25	**Einstieg und Projektstand** 0–5 Einstieg 5–25 Berichte aus der Zwischenphase	**25–45**	**Überblicksvortrag** Gesamtprozess und Weg zur Sample-Feature-Matrix	**45–90**	**Teamarbeit · Block 1** EDA vertiefen und Datenregeln dokumentieren
90–100	**Pause** 10 Minuten	**100–125**	**Vertiefung und Ausblick** tsfresh, DTW, Transformation und nächste Zwischenphase	**125–180**	**Teamarbeit · Block 2** Features umsetzen und Modellierung vorbereiten

Projektstand aus der Zwischenphase

4 Teams × 5 Minuten · jeweils 4 Minuten Bericht und 1 Minute Rückfragen.

01	**Datenverständnis**	Datenstruktur, Zeitbasis und relevante Datenprobleme zusammenfassen.
02	**EDA-Ergebnis**	Einen aussagekräftigen Plot mit Beobachtung und Hypothese erläutern.
03	**Arbeitsstand**	Erledigte Aufgaben, offene Fragen und den nächsten Prüfschritt benennen.

> Die offenen Fragen aus den Berichten bestimmen die Schwerpunkte der Teamarbeit.

Grundstruktur einer datengetriebenen Anwendung

01	**Dateneingang**	Messwerte, Zeitstempel und Kontext einlesen.	Rohdaten + Datenbeschreibung
02	**Datenvorverarbeitung**	Zeitbasis, Datenqualität und Fenster festlegen.	Gültige Signalabschnitte
03	**Feature-Erzeugung**	Signale in numerische Merkmale überführen.	Featurematrix X
04	**Feature-Transformation**	Features imputieren, skalieren oder auswählen.	Modellfähige Merkmale Z
05	**Modelltraining**	Modellparameter auf Trainingsdaten bestimmen.	Angepasstes Modell
06	**Modellevaluation**	Leistung auf getrennten Daten und Fehler prüfen.	Vergleich + Einsatzgrenzen
07	**Modellbereitstellung**	Verarbeitungskette ausführen und überwachen.	Vorhersagen / Scores im Betrieb

Hervorgehoben: Schwerpunkt Session 2 · 01: Session 1 · 05–06: Session 3 · 07: Session 4

Quellen: scikit-learn · Leakage and preprocessing

> Datentrennung vorab festlegen. Gelernte Verarbeitungsschritte innerhalb der Trainingsdaten anpassen.

ML-Grundlagen · Samples, Features und Zielwerte

X: 6 Samples × 3 Features

	f_1	f_2	f_3
x_1	10	2	5
x_2	12	4	7
x_3	9	3	6
x_4	14	5	8
x_5	11	2	6
x_6	13	4	9

y: 6 Zielwerte

0
1
0
1
0
1

Featurevektor x_2 = [12, 4, 7]

Train-Test-Split: gleiche Zeilen in X und y

Training · Modell anpassen

(X_train, y_train) → f

Test · neue Samples bewerten

X_test → f → ŷ_test

ŷ_test mit y_test vergleichen

Trennregel passend zu Zeit und Gruppen wählen.

Sample-Feature-Matrix X

n Zeilen = Samples; p Spalten = Features. Hier: X mit Form 6 × 3.

Featurevektor x_i

Alle p Merkmale eines Samples i; als Zeile in X gespeichert.

Zielvektor y

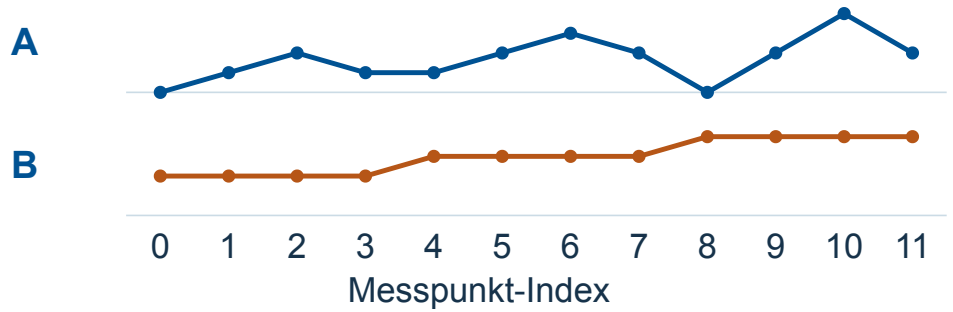
Ein Zielwert y_i je Sample: Klasse oder Zahl. Zuordnung zu X erhalten.

Synthetisches Beispiel für überwachtes Lernen · Klassen 0/1 · Testdaten für die abschließende Bewertung zurückhalten.

Quellen: scikit-learn · Samples, features and targets · scikit-learn · Cross-validation

Von Zeitreihen zur Sample-Feature-Matrix

Zeitreihen: 12 Zeitpunkte × 2 Kanäle

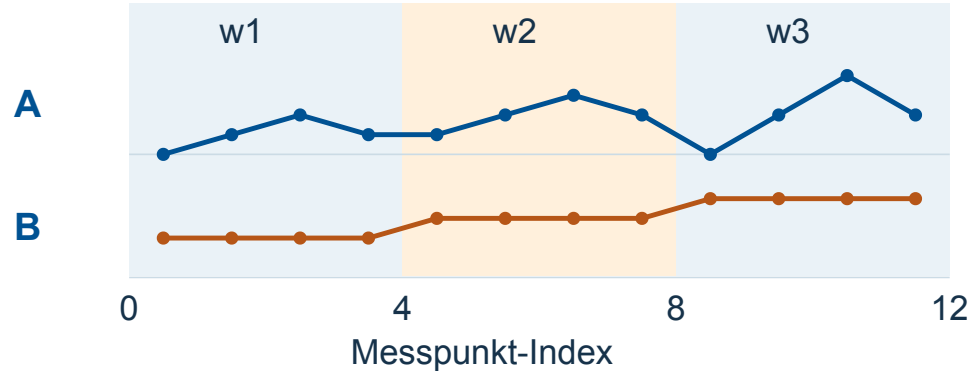


Synthetische Lehrbeispiele · relative Werte und vereinfachte Zeitachsen.

> Wie würdet ihr aus diesen Zeitreihen eine Sample-Feature-Matrix erstellen?

Von Zeitreihen zur Sample-Feature-Matrix

Zeitreihen: 12 Zeitpunkte × 2 Kanäle



je Fenster

X: 3 Samples × 4 Features

ID	A.mean	A.std	B.mean	B.std
w1	1	0,71	2	0
w2	2	0,71	3	0
w3	2	1,41	4	0

ID ≠ Feature

Separate Metadaten: record_id, Fensterstart, Fensterende

Sample = Beobachtung

Hier: ein abgeschlossenes Fenster einer Messung.

Feature = Spalte

Dieselbe Definition, Einheit und Reihenfolge für jede Zeile.

Zielwert y separat

Bei überwachtem Lernen gehört zu jeder Zeile ein passender Zielwert.

Synthetische Lehrbeispiele · relative Werte und vereinfachte Zeitachsen.

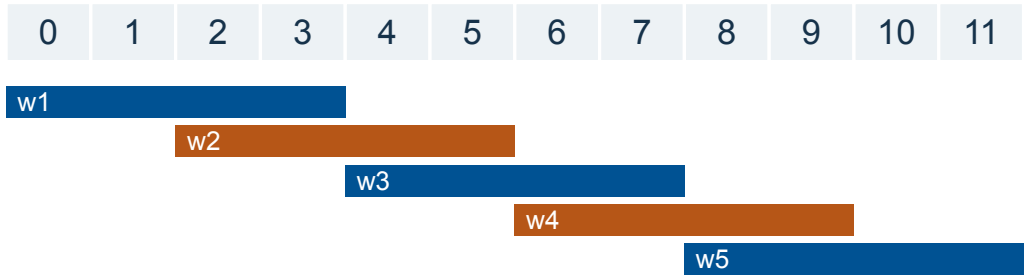
Quellen: scikit-learn · Samples, features and targets

Zeitreihen strukturieren

Welche Entscheidungen müsst ihr treffen, bevor aus einer Zeitreihe mehrere Samples werden?

Fensterlänge und Schrittweite bestimmen die Zeilen

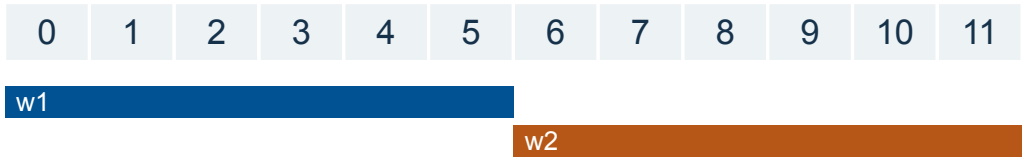
Länge 4 · Schrittweite 2



5 Matrixzeilen

Überlappende Fenster teilen Rohwerte.

Länge 6 · Schrittweite 6



2 Matrixzeilen

Längere Fenster fassen mehr Verlauf zusammen.

Zeitskala

Fensterdauer an Ereignisdauer und Entscheidungsrhythmus ausrichten.

Informationsverlust

Große Fenster können kurze Ereignisse abschwächen.

Abhängigkeit

Mehr Fenster erzeugen zusätzliche, oft stark ähnliche Zeilen.

Synthetische Lehrbeispiele · relative Werte und vereinfachte Zeitachsen.

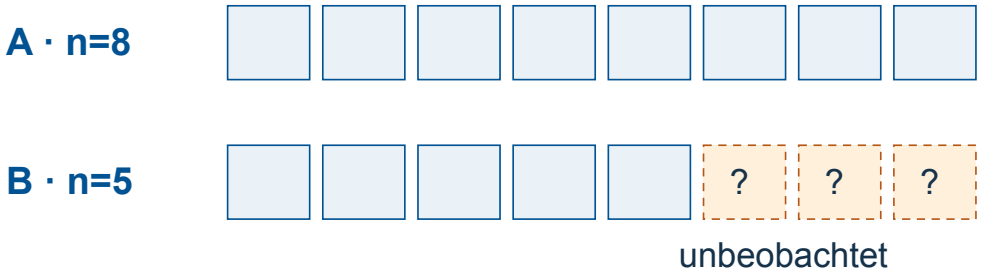
Quellen: pandas · Rolling windows

Messungen vergleichen

Wie würdet ihr Messungen mit unterschiedlicher Länge und unterschiedlichen Zeitstempeln vergleichbar machen?

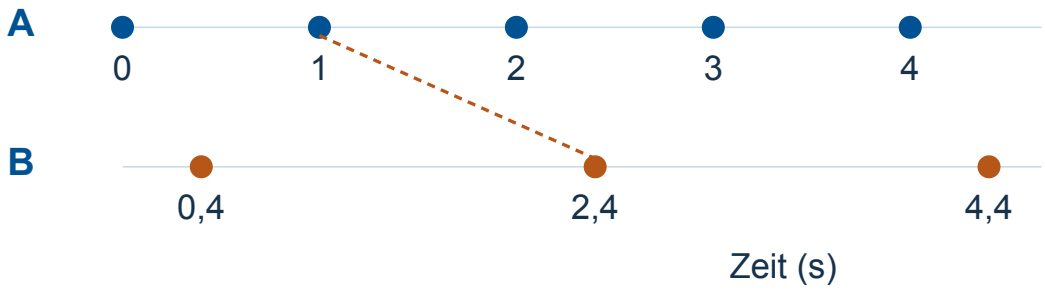
Messlängen und Sensorzeitachsen unterscheiden sich

Unterschiedliche Messlängen



Fenster / Zyklus / Gesamtversuch festlegen.

Unterschiedliche Zeitstempel



Über Zeit und Versuch zuordnen.

Länge

Abschneiden, Auffüllen und Aggregation verändern die Aussage.

Synchronisation

Gleiche Zeilennummern können verschiedene Zeitpunkte beschreiben.

Regel

Zeitbasis, Toleranz und verfügbare Werte dokumentieren.

Synthetische Lehrbeispiele · relative Werte und vereinfachte Zeitachsen.

Quellen: pandas · Time-based joins · pandas · Rolling windows

Unvollständige Messungen

Ein Sensor fällt zeitweise aus. Wie würdet ihr mit den betroffenen Samples umgehen?

Unvollständige Fenster erzeugen ungültige Features

Beispiel: vollständige B-Fenster erforderlich

ID	A.mean	B.mean	B.std	B.coverage
w1	1	2	0	100 %
w2	2	NaN	NaN	50 %
w3	2	NaN	NaN	0 %
w4	3	4	0	100 %

Lücken → ungültige Zellen

Konstanter Kanal → std = 0

Abdeckung separat mitführen

Ausschluss oder begründete Imputation

NaN

Fehlende Messungen sind von gültigen Nullwerten zu unterscheiden.

Konstante Spalten

Bei Standardisierung ist eine Regel für σ = 0 erforderlich.

Nachvollziehbarkeit

Maske, Abdeckung und verworfene Fenster protokollieren.

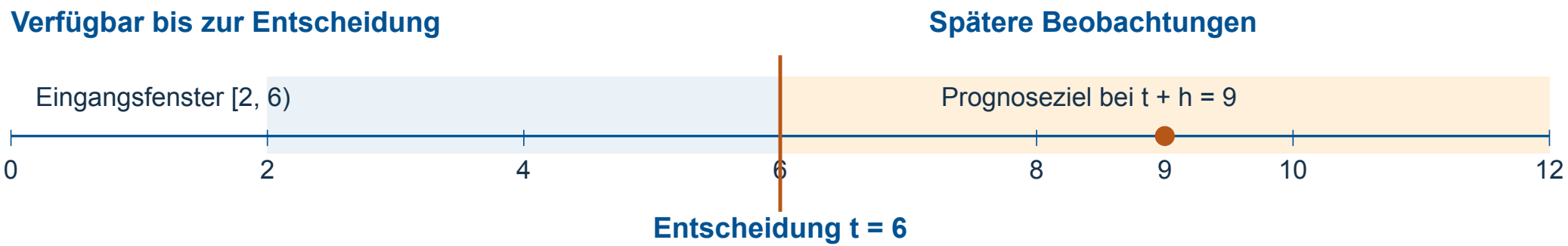
Synthetische Lehrbeispiele · relative Werte und vereinfachte Zeitachsen.

Quellen: scikit-learn · Leakage and preprocessing · pandas · Interpolation

Vorhersagen vorbereiten

Wir treffen bei t = 6 eine Vorhersage für t = 9. Welche Informationen dürfen in das Sample eingehen?

Features und Zielwerte brauchen einen Zeitbezug



X-Zeile: Features aus dem Eingangsfenster

y: separat zugeordnetes Ziel

Schlüssel: Versuch + Fensterende + Zieldefinition

Verfügbarkeit

Eingangsdaten müssen zum gewählten Entscheidungszeitpunkt vorliegen.

Zieldefinition

Prognosehorizont oder Ereigniszuordnung fachlich festlegen.

Zuordnung

X, y und Gruppen über stabile IDs gemeinsam filtern und sortieren.

Synthetisches Prognosebeispiel · bei Anomalieerkennung sind bestätigte Labels optional und gesondert zu prüfen.

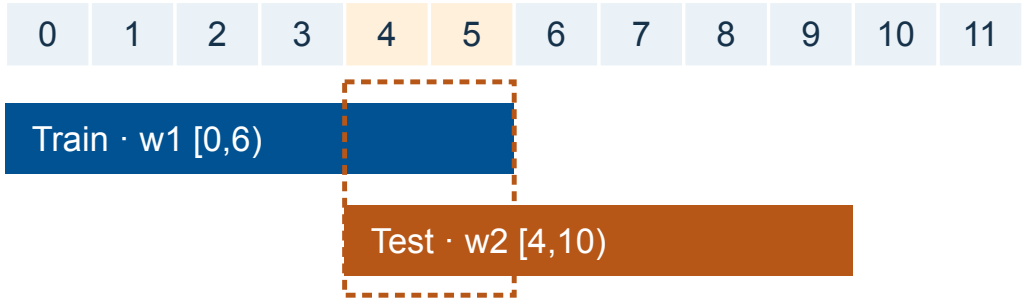
Quellen: scikit-learn · Samples, features and targets · scikit-learn · Lagged features

Übertragbarkeit prüfen

Wie würdet ihr Trainings- und Testdaten trennen, um die Übertragbarkeit auf neue Messungen zu prüfen?

Überlappende Fenster beeinflussen den Modellvergleich

Ein Versuch: Fenster teilen Rohdaten



Gemeinsame Messpunkte

Ziel: auf neue Versuche übertragen

Versuch A → Training

Versuch B → Test

Trennung nach Versuch vor der Fensterbildung

Abhängigkeit

Zufällige Zeilensplits können fast gleiche Fenster auf beide Seiten verteilen.

Einsatzbezug

Für neue Versuche nach Versuch gruppieren; für Zukunftsprognosen zeitlich trennen.

Trainingsgrenze

Imputation, Skalierung und Auswahl innerhalb der Trainingsdaten anpassen.

Synthetische Lehrbeispiele · relative Werte und vereinfachte Zeitachsen.

Quellen: scikit-learn · Cross-validation · scikit-learn · Leakage and preprocessing

Teamarbeit · EDA und Datenqualität

45 min

Bearbeitet die offenen Fragen aus eurem Statusbericht. Setzt eigene Prioritäten und arbeitet bis zur Pause durchgehend im Team.

01	**Analyse vertiefen**	Messungen vergleichen; Trends, wiederkehrende Muster und Streuung untersuchen.
02	**Datenregeln prüfen**	Zeitbasis, Fehlwerte und Auffälligkeiten prüfen; Entscheidungen begründen.
03	**Ergebnisse sichern**	Sample definieren: Messung, Fenster, Zeitpunkt und gemeinsame Feature-Spalten.

> Bis zur Pause: EDA-Notebook, zwei Plots und ein Schema für die Sample-Feature-Matrix.

90–100 min

Pause

Anschließend: technische Vertiefung und Ausblick auf die nächste Zwischenphase.

Lasst einen Plot geöffnet, an dem ihr weiterarbeiten möchtet.

10 min

Die Featurematrix weiterverarbeiten

Eine Featurematrix enthält Lücken und stark unterschiedliche Größenordnungen. Wie würdet ihr sie für ein Modell vorbereiten?

Featurematrix für das Modell vorbereiten

Entscheidung	Anpassung auf Trainingsdaten	Anwendung auf neue Daten
`Imputation`	Ersatzwerte und gültige Spalten festlegen	Gespeicherte Ersatzwerte verwenden
`Skalierung`	μ und σ je Feature bestimmen	z = (x − μ_train) / σ_train
`Auswahl / PCA`	Spaltenauswahl oder Projektion lernen	Identische Spalten / Projektion nutzen

Quellen: scikit-learn · Leakage and preprocessing

> fit auf Trainingsdaten; transform auf Validierungs-, Test- und Betriebsdaten. Alle Schritte gemeinsam versionieren.

tsfresh · Automatische Feature-Erzeugung

tsfresh berechnet ausgewählte statistische Merkmale für jede definierte Zeitreihe oder jedes Fenster.

01	**Eingang**	Tabelle mit id, time und value. Jede id bezeichnet hier ein vorbereitetes Fenster.
02	**Berechnung**	Feature-Funktionen auswählen, z. B. mean, standard_deviation und maximum.
03	**Ausgang**	Eine Zeile pro Fenster; eine Spalte pro berechnetem Feature.

Quellen: tsfresh · Feature extraction · tsfresh · Data formats

> Synthetisches Fenster [0, 1, 2, 1] → mean = 1 · std ≈ 0,707 · maximum = 2

tsfresh · Fenster in eine Featurematrix überführen

```
from tsfresh import extract_features
fc = {
    'mean': None, 'standard_deviation': None,
    'maximum': None,
}
X = extract_features(
    windows, column_id='id', column_sort='time',
    column_value='value', default_fc_parameters=fc,
    n_jobs=0, disable_progressbar=True,
)
```

windows

Vorbereitete Tabelle: id, time, value; endliche Werte innerhalb gültiger Fenster.

Ergebnis

X.index enthält Fenster-IDs. Spalten heißen z. B. value__mean.

Prüfung

Ein Fenster manuell nachrechnen; ungültige Feature-Werte gezielt behandeln.

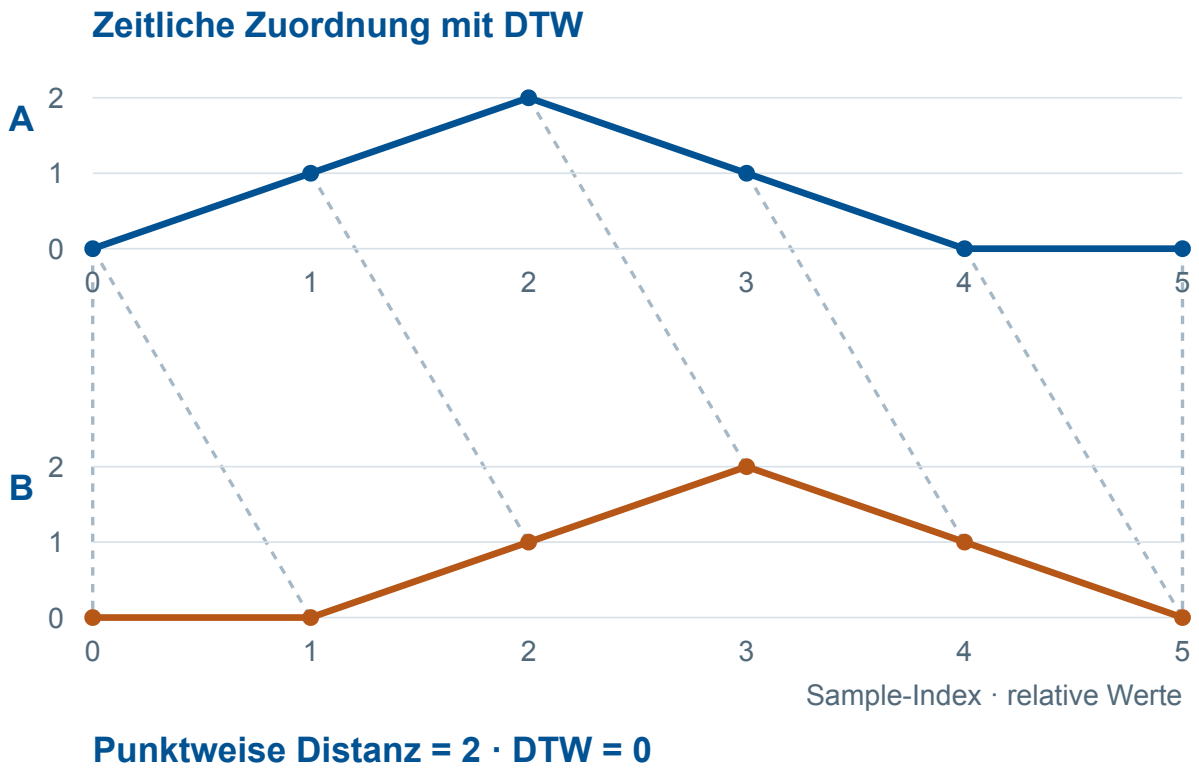
Ausführbares synthetisches Beispiel: feature-methods.py · keine Spaltenvorgabe für den Projektdatensatz.

Quellen: tsfresh · Extraction settings · tsfresh · Data formats

Ähnliche Signalverläufe

Zwei Signale verlaufen ähnlich, sind aber zeitlich versetzt oder unterschiedlich schnell. Wie würdet ihr ihre Ähnlichkeit bewerten?

Dynamic Time Warping · Verläufe vergleichen



Fragestellung

Gleiche Form bei unterschiedlichem zeitlichem Ablauf?

Zuordnung

DTW ordnet Werte entlang eines monotonen Pfads zu; Wiederholungen sind möglich.

Ergebnis

Die Distanz zur Referenz kann ein Feature oder ein Anomalie-Score sein.

Synthetisches Beispiel · Punktdistanz: $\sqrt{\Sigma}(A_i - B_i)^2 = 2$ · DTW: $\sqrt{\Sigma}$ Pfadkosten = 0 · Radius 1

Quellen: tslearn · Dynamic Time Warping

DTW berechnen und die Anpassung begrenzen

```
from tslearn.metrics import dtw_path
a = [0, 1, 2, 1, 0, 0]
b = [0, 0, 1, 2, 1, 0]
path, distance = dtw_path(
    a, b, global_constraint='sakoe_chiba',
    sakoe_chiba_radius=1,
)
# distance == 0.0
```

Zeitliche Grenze

Radius 1 erlaubt im Beispiel $|i - j| \leq 1$.

Fachliche Grenze

Zeitfehler können durch Warping verdeckt werden; Amplitude und Skalierung prüfen.

Referenzwahl

Referenz und Parameter auf Entwicklungsdaten festlegen; getrennt evaluieren.

Kostenrekursion: $C_{i\,j} = (a_i - b_j)^2 + \min(C_{i_{-1},\,j}, C_{i,\,j_{-1}}, C_{i_{-1},\,j_{-1}})$ · DTW = $\sqrt{C_{nm}}$

Quellen: tslearn · dtw_path

DTW passt, wenn Zeitvariation zulässig ist. Bei zeitkritischen Fehlern die zeitliche Abweichung gesondert bewerten.

Referenzdaten, Score und Schwelle

01

Normalbereich

Welche Versuche gelten begründet als vergleichbar und unauffällig?

02

Score

In der Demo: std_8 pro gültigem Zeitpunkt.

03

Regel

Score oberhalb einer vorab gewählten Schwelle markieren.

Quellen: scikit-learn · Novelty detection

Ohne bestätigte Referenz bleiben die Ergebnisse explorative Kandidaten zur Prüfung.

Arbeitsauftrag bis Session 3

Beginnt die Umsetzung im folgenden Arbeitsblock und führt sie bis Session 3 eigenständig fort.

01	**05 · Baseline trainieren**	Feature- und Transformationsregeln festhalten; eine einfache Baseline auf Trainingsdaten anpassen.
02	**06 · Varianten evaluieren**	Einen Ansatz gezielt verändern; dieselbe Datentrennung für den Vergleich nutzen.
03	**07 · Einsatz vorbereiten**	Datenvertrag, gespeicherte Verarbeitungsschritte und Überwachungsbedarf beschreiben.

> Zu Session 3: reproduzierbare Verarbeitung, Baseline-Vergleich und Review-Protokoll. Deployment wird in Session 4 vertieft.

Teamarbeit · Features und Modellierungsplan

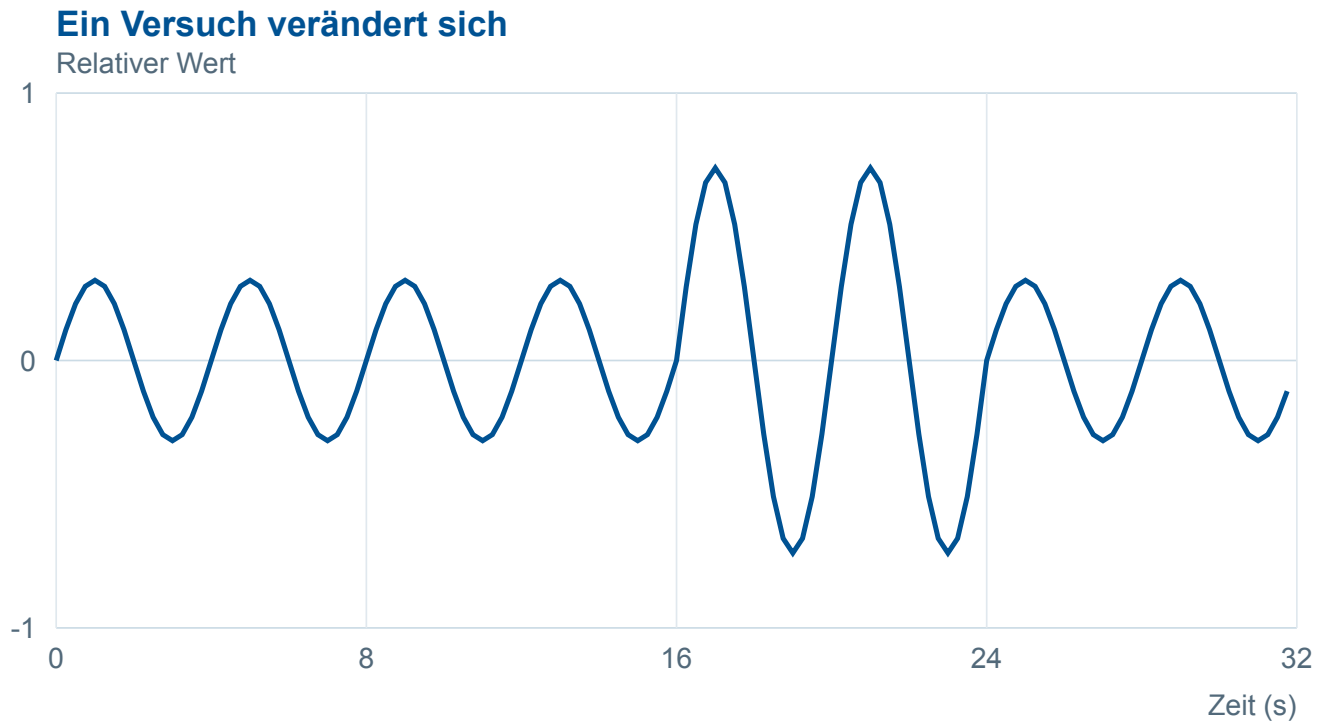
55 min

Setzt eure Feature-Idee um und bereitet die erste Modellierungsiteration vor. Arbeitet bis zum Ende der Session im Team.

01	**Features umsetzen**	Kleine Sample-Feature-Matrix berechnen. Eine Zeile am Signal nachrechnen; IDs sichern.
02	**Transformation prüfen**	Fehlende Features und Skalierung prüfen; Anpassung auf Trainingsdaten begrenzen.
03	**Weiterarbeit planen**	Aufgaben, Verantwortliche, Abnahmekriterien und einen Review-Termin dokumentieren.

> Zum Ende: Feature-Code, geprüfte Transformationsregel und Plan für Training und Evaluation.

Ab Sekunde 16 schwingt das Signal stärker



Beobachten

Die Ausschläge werden größer. Der Mittelwert bleibt ähnlich.

Erklären

Belastung? Veränderte Struktur? Messfehler?

Prüfen

Welche zusätzliche Information trennt diese Erklärungen?

Synthetisches Lehrbeispiel · relative Einheiten · unabhängig vom Projektdatensatz.

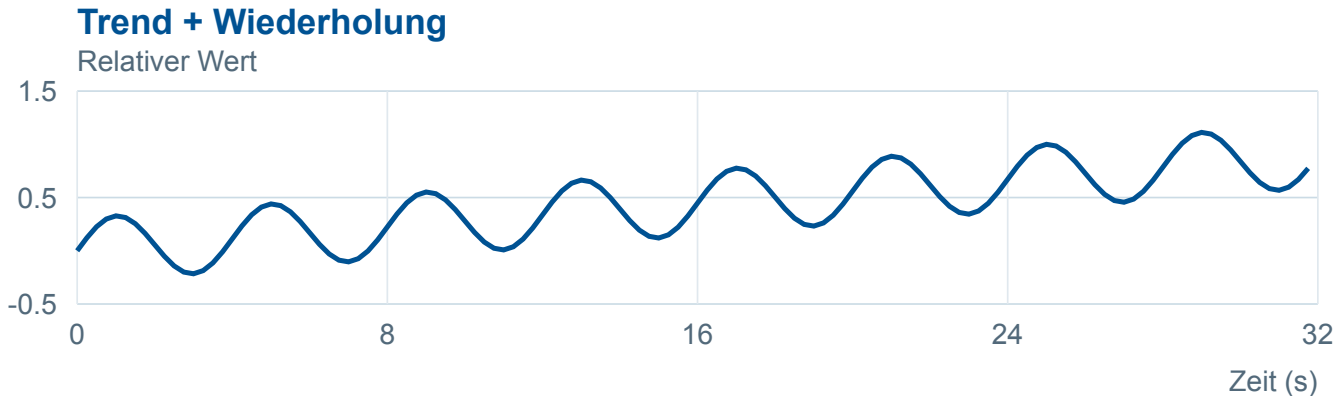
Darstellungen für die Zeitreihenanalyse

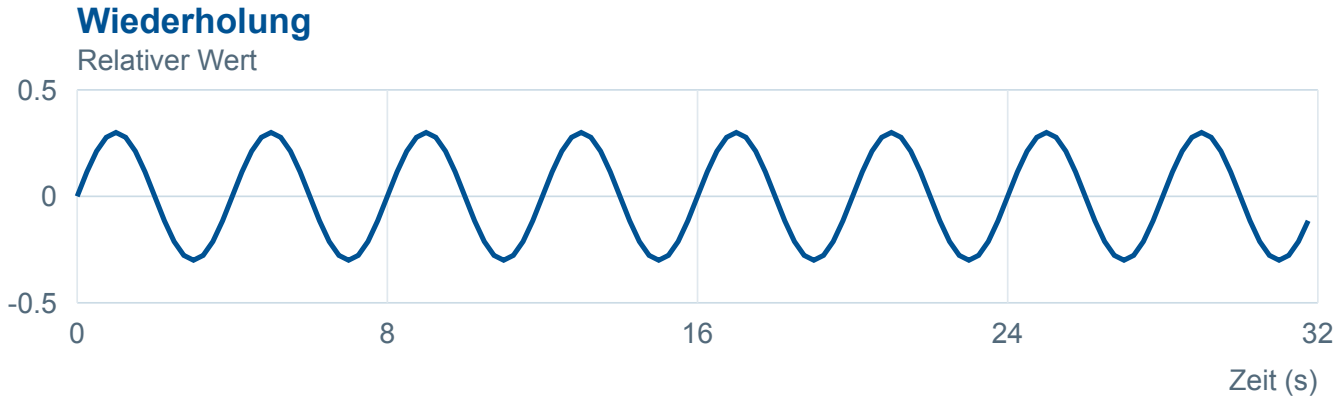
01	**Zeitverlauf**	Wann beginnt die Änderung? Welche Phase liegt davor und danach?
02	**Verteilung**	Wie ändern sich Lage und Streuung innerhalb vergleichbarer Phasen?
03	**Versuchsvergleich**	Bleibt das Muster bei gleichem Ablauf oder Betriebszustand bestehen?

Quellen: FPP3 · Time series graphics

> Jeder Plot beantwortet eine konkrete Frage und führt zu einem nächsten Prüfschritt.

Trend, Periodik und Saisonalität





Trend

Das Niveau verändert sich über den beobachteten Zeitraum.

Periodik

Ein Muster kehrt in ähnlichen Abständen wieder.

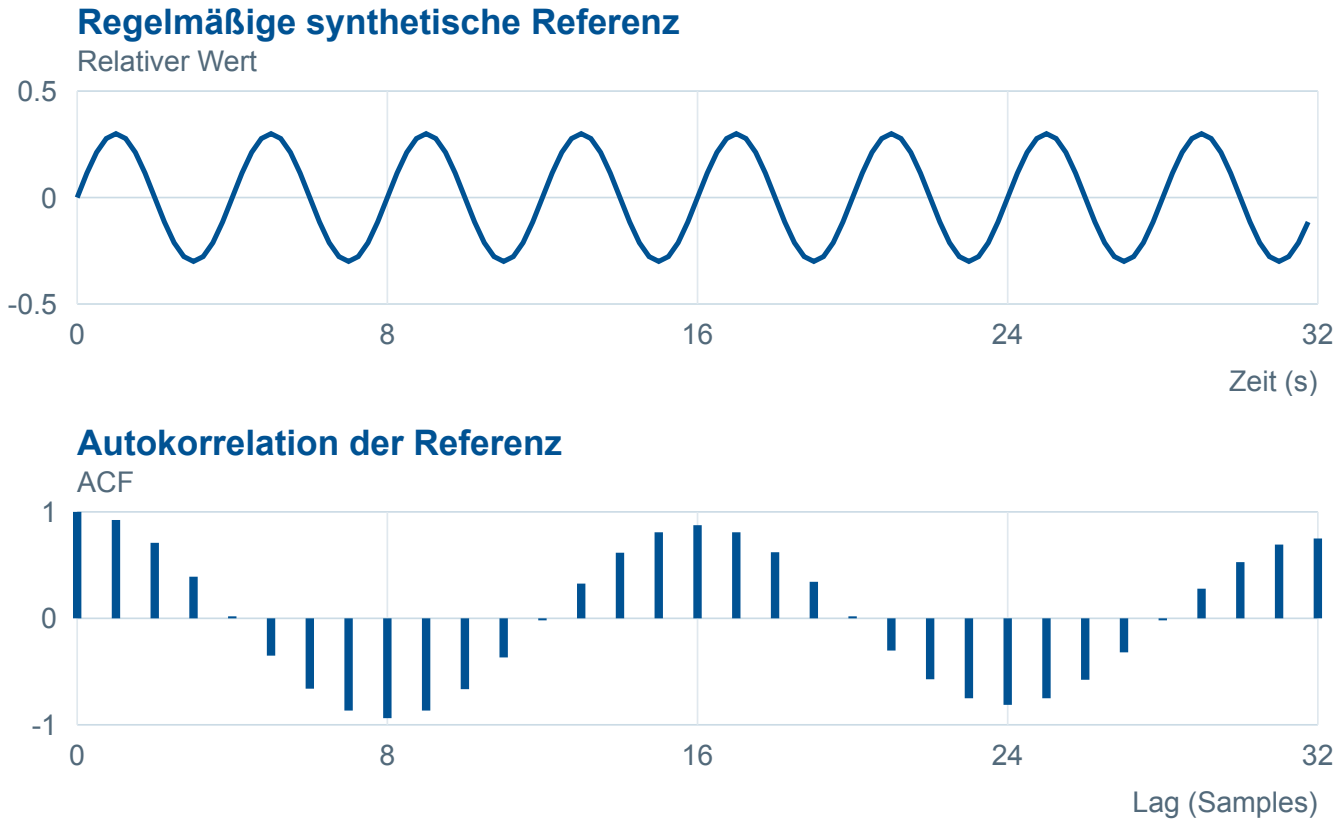
Saisonalität

Kalendergebundene Wiederholung braucht passende Zeitabdeckung.

Synthetisches Lehrbeispiel · relative Einheiten · unabhängig vom Projektdatensatz.

Quellen: FPP3 · Time series graphics

Autokorrelation und zeitliche Abhängigkeit



Lag k

Vergleicht eine Reihe mit ihrer um k Schritte verschobenen Version.

ACF bei 16

Die Demo wiederholt sich nach 16 Samples = 4 Sekunden.

Einordnen

Trend, kurze Reihen und Zustandswechsel beeinflussen die ACF.

Synthetische Referenz · ACF aus den gezeigten 128 Werten berechnet.

Quellen: FPP3 · Time series graphics

Zeitachse und Abtastung prüfen

01	**Reihenfolge**	Pro Versuch sortieren; doppelte Zeitpunkte und Rücksprünge klären.
02	**Abstände**	Verteilung von Δt prüfen. Ein Lag von 8 Zeilen hat nur bei konstantem Δt eine feste Dauer.
03	**Grenzen**	Versuche und Unterbrechungen getrennt behandeln; Fenster innerhalb gültiger Abschnitte bilden.

Quellen: pandas · Rolling windows · pandas · Series.shift

> Ein regelmäßiges Raster und eine dokumentierte Zeitbasis machen Features interpretierbar.

Regeln für fehlende und auffällige Werte

Befund	Mögliche Entscheidung	Beleg sichern
Zeitbasis	Sortierung, Abstände und Versuchstrennung prüfen	Zeitstempel + Fensterdefinition
Kurze Lücke	Begrenzt füllen, falls fachlich plausibel	Rohwerte + Fehlwertmaske
Lange Unterbrechung	Abschnitt trennen; Fenster verwerfen	Dauer + Ausschlussgrund
Einzelne Spitze	Ursache prüfen; zunächst markieren	Original + Kontext
Neues Niveau	Betriebsphase und Kalibrierung prüfen	Vorher/Nachher-Vergleich

Quellen: NIST · Detection of outliers · pandas · Interpolation

Features aus Hypothesen ableiten

Vermutetes Muster	Kandidat	Wichtige Grenze
`Niveau verschiebt sich`	Rolling Mean / Median	Fenster kann Übergang verwischen
`Streuung nimmt zu`	Rolling Std	Spitzen und Betriebszustand wirken mit
`Dynamik verändert sich`	Lag-Wert / Differenz	Zeitraster bestimmt Bedeutung
`Schwingung wird stärker`	RMS oder Streuung	RMS reagiert auch auf Offset

Quellen: pandas · Rolling windows

> Unsere Demo-Hypothese führt zur lokalen Standardabweichung.

Datenverfügbarkeit zum Entscheidungszeitpunkt

Konvention: Wir entscheiden bei t, bevor x_t verfügbar ist.

Größe bei t	Verwendete Werte	Beispiel bei t = 4
`x: 2, 4, 6, 8, 10`	Index: 0, 1, 2, 3, 4	x_4 = 10 ist noch unbekannt
`lag₁(t)`	x_{t-1}	8
`mean₃(t)`	x_{t-3} … x_{t-1}	(4 + 6 + 8) / 3 = 6
`std₃(t), ddof = 0`	x_{t-3} … x_{t-1}	√(8/3) ≈ 1,63

Quellen: pandas · Series.shift · scikit-learn · Lagged features

> Für eine Diagnose nach Eingang von x_t darf x_t Teil des Fensters sein. Zeitpunkt und Ziel explizit festlegen.

Lag- und Rolling-Features mit pandas

```
# x: one regular, ordered recording
past = x.shift(1)
features = pd.DataFrame({
    'lag_1': past,
    'mean_8': past.rolling(8, min_periods=8).mean(),
    'std_8': past.rolling(8, min_periods=8).std(ddof=0),
})
valid = features.notna().all(axis=1)
```

Fenster

8 Samples bei 4 Hz: 2 Sekunden Demo-Kontext.

Abdeckung

Die ersten 8 Zeilen und unvollständige Fenster bleiben ungültig.

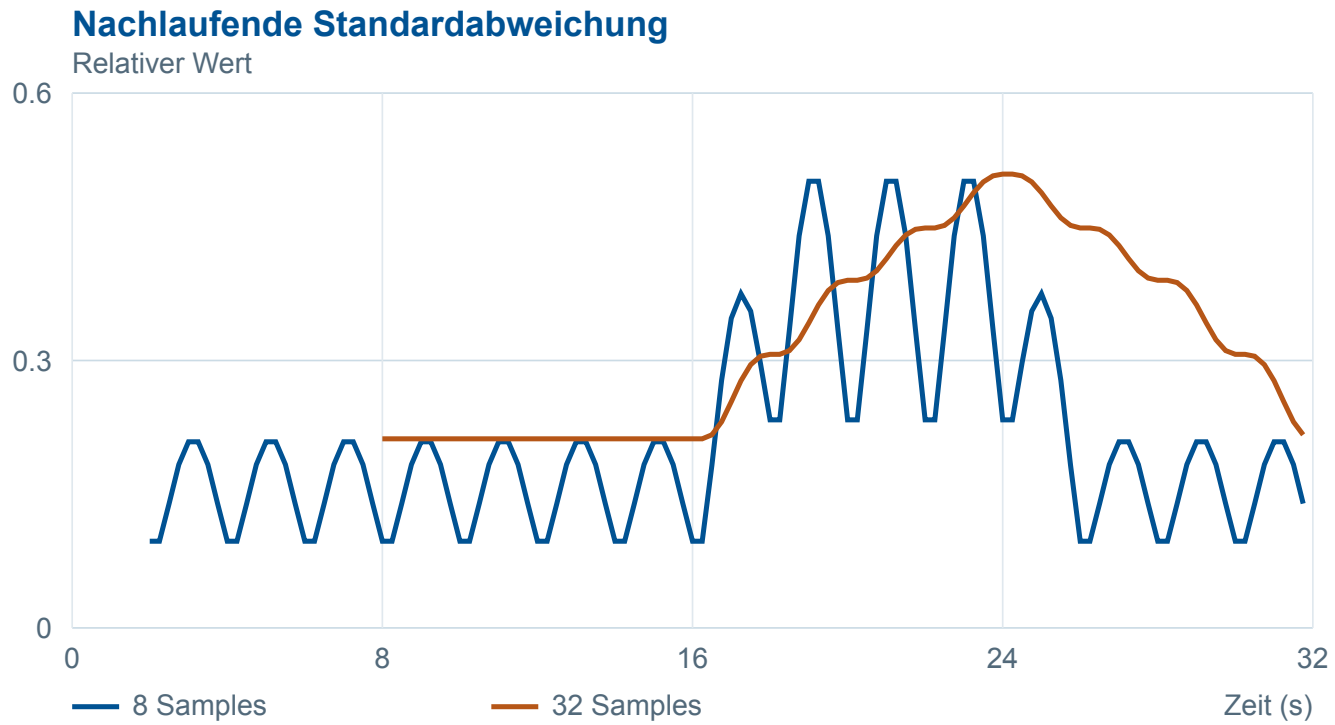
Pro Versuch

Den Code für jeden gültigen Abschnitt getrennt ausführen.

Python · pandas · Demo-Parameter; Sampling und Fenster für das Projekt begründen.

Quellen: pandas · Rolling windows · pandas · Series.shift

Einfluss der Fensterlänge



8 Samples

Reagiert rascher; variiert mit der Phase des Signals.

32 Samples

Mittelt über mehr Zyklen; Übergänge wirken länger nach.

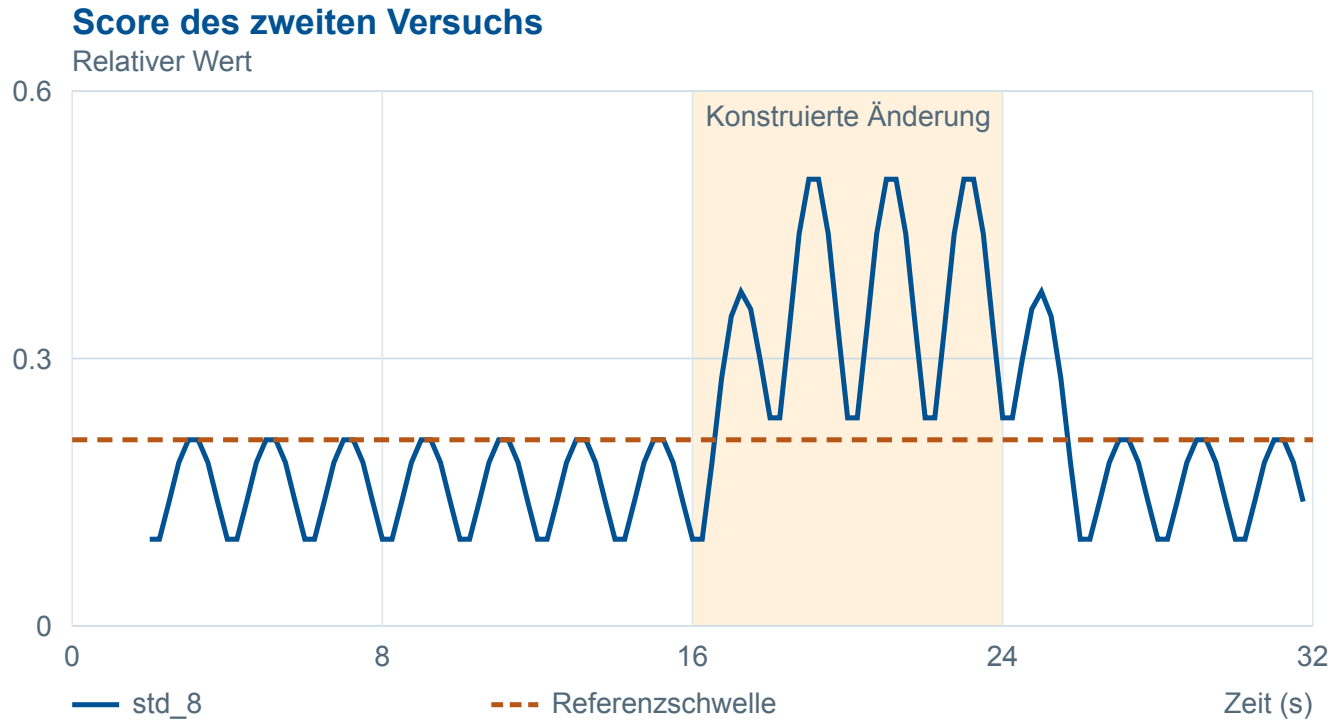
Wahl begründen

Ereignisdauer, Abtastrate und zulässige Verzögerung verbinden.

Synthetisches Lehrbeispiel · relative Einheiten · unabhängig vom Projektdatensatz.

Quellen: pandas · Rolling windows

Anwendung einer schwellenbasierten Baseline



Kalibrieren

Demo-Schwelle: 95. Perzentil der gültigen Referenz-Scores.

Anwenden

Dieselbe Regel auf den zweiten Demo-Versuch anwenden.

Zurück zum Signal

Markierte Abschnitte im Rohsignal und Kontext prüfen.

Synthetisches Lehrbeispiel · relative Einheiten · unabhängig vom Projektdatensatz.

Quellen: scikit-learn · Novelty detection

Ein erhöhter Score begründet eine Untersuchung. Die Ursache bleibt eine fachliche Frage.

Datentrennung für den Modellvergleich

01	**Versuche trennen**	Fenster desselben Versuchs bleiben zusammen. Gruppierung am späteren Einsatz ausrichten.
02	**Zeit beachten**	Für zukünftige Einsätze auf späteren Daten prüfen. Überlappung an Grenzen vermeiden.
03	**Regeln festlegen**	Features und Schwellen auf Entwicklungsdaten wählen. Den finalen Test reservieren.

Quellen: scikit-learn · Cross-validation · scikit-learn · Lagged features

> Heute: Trennregel dokumentieren. Session 3: Backtesting, Metriken und Fehleranalyse.

Aufgaben und Abnahmekriterien

Aufgabe	Verantwortung → Review	Fertig, wenn …
Zeitbasis und Referenz prüfen	A → B	Prüfbericht + begründete Versuchsliste vorliegen
Feature-Funktion bauen	B → C	Fenster, Einheiten und Fehlwertregel geprüft sind
Baseline anwenden	C → A	Score-Plot und feste Schwelle reproduzierbar sind
Gemeinsames Review	Alle	Befund, Grenze und nächster Versuch feststehen

Quellen: Schwaber & Sutherland · Scrum Guide

> Gemeinsames Done: frischer Lauf, dokumentierte Parameter, geprüfter Plot und festgehaltene Grenzen.

Beobachtungen und Hypothesen dokumentieren

Schritt	Eintrag im EDA-Log
`Beobachtung`	Größere Ausschläge im Demo-Intervall 16–24 s.
`Hypothese`	Die lokale Streuung bildet diese Änderung ab.
`Alternative`	Ein anderer Betriebszustand erzeugt dieselbe Änderung.
`Nächster Check`	Streuung in vergleichbaren Phasen und Versuchen vergleichen.

> Ein EDA-Ergebnis verbindet Beleg, Erklärung und Konsequenz für die Modellierung.

Feature-Definition dokumentieren

Feld	Demo-Entscheidung
Zweck	Erhöhte lokale Streuung sichtbar machen
Eingang	Ein gültiger, regelmäßiger Signalabschnitt
Zeitbezug	8 frühere Werte; x_t bleibt ausgeschlossen
Fehlwerte	Nur vollständige Fenster auswerten
Ausgang	std_8 in derselben Einheit wie das Eingangssignal

> Eine andere Person im Team kann das Feature mit denselben Regeln reproduzieren.

Iterationsziel, Abstimmung und Review

01

Iterationsziel

Eine begründete Baseline auf getrennten Versuchen reproduzieren.

02

Kurze Abstimmung

Was ist fertig? Was blockiert? Welche Übergabe braucht ihr?

03

Review

Code ausführen, Befunde prüfen, nächste Änderung wählen.

Quellen: Schwaber & Sutherland · Scrum Guide

Teamregel: Eine Person verantwortet die Aufgabe; eine zweite prüft das Ergebnis.

Ergänzung: Baselines für Prognosen

Für Energie- oder Verbrauchsreihen: Ziel ist der nächste Wert x_t.

Baseline	Vorhersage bei t	Voraussetzung
Naiv	$\hat{x}_t = x_{t-1}$	Letzter Wert ist bekannt
Saisonal naiv	$\hat{x}_t = x_{t-m}$	Periode m ist fachlich begründet
Mittelwert	$\hat{x}_t$ = Mittelwert der Trainingswerte	Referenzzeitraum ist geeignet

Quellen: FPP3 · Forecasting toolbox

> Prognosen: Fehler zu später beobachteten Werten messen. Anomalien: markierte Ereignisse fachlich prüfen.

Arbeitsblatt · EDA → Feature → Iteration

Befund

Plot / Messung / Abschnitt
Beobachtung: …

Hypothese

Erklärung: …
Alternative: …

Datenregel

Zeitbasis: …
Lücken / Ausreißer: …

Feature

Eingang / Einheit: …
Fenster / Zeitpunkt: …

Baseline

Referenz: …
Score / Regel: …

Vergleich

Trennregel: …
Prüfkriterium / Grenze: …

Iterationsziel

Prüfbares Ergebnis: …
Termin: …

Aufgaben

Owner / Reviewer: …
Abhängigkeit: …

Done

Frischer Lauf / Plot: …
Parameter / Grenzen: …

Review

Termin / Teilnehmende: …
Nächste Entscheidung: …

Session 2 · Von Signalen zu Modellmerkmalen

02

Vorverarbeitung

Datenqualität und Zeitbasis prüfen. Beobachtungseinheit und Fenster definieren.

03

Feature-Erzeugung

Pro Sample fachlich begründete Merkmale berechnen und in gemeinsamen Spalten ablegen.

04

Transformation

Fehlende Features behandeln, Größenordnungen anpassen und Auswahl begründen.

Block 1: Datenbasis und Feature-Idee. Block 2: Featurematrix, Transformationsregel und Modellierungsplan.

Feature-Erzeugung und Transformation

Schritt	Eingang → Ausgang	Beispiel
03 · Erzeugung	Signalabschnitt → Merkmale	Mittelwert, Streuung, Maximum
03 · Erzeugung	Signalabschnitt → Trend	Steigung über das definierte Fenster
04 · Transformation	Merkmale → skalierte Merkmale	$z = (x - \mu_{train}) / \sigma_{train}$
04 · Transformation	Viele Merkmale → Teilmenge	Konstante oder redundante Features prüfen

Die Feature-Definition bestimmt die gemessene Eigenschaft. Die Transformation bestimmt ihre Darstellung für das Modell.

Quellen · Analyse & Daten

FPP3 · Time series graphics

https://otexts.com/fpp3/graphics.html

NIST · Detection of outliers

https://www.itl.nist.gov/div898/handbook/eda/section3/eda35h.htm

pandas · Interpolation

https://pandas.pydata.org/docs/reference/api/pandas.Series.interpolate.html

pandas · Rolling windows

https://pandas.pydata.org/docs/reference/api/pandas.Series.rolling.html

pandas · Series.shift

https://pandas.pydata.org/docs/reference/api/pandas.Series.shift.html

Quellen · Baselines & Zusammenarbeit

FPP3 · Forecasting toolbox

https://otexts.com/fpp3/toolbox.html

scikit-learn · Lagged features

https://scikit-learn.org/stable/auto_examples/applications/plot_time_series_lagged_features.html

scikit-learn · Cross-validation

https://scikit-learn.org/stable/modules/cross_validation.html

scikit-learn · Novelty detection

https://scikit-learn.org/stable/modules/outlier_detection.html

Schwaber & Sutherland · Scrum Guide

https://scrumguides.org/scrum-guide.html

Quellen · Sample-Feature-Matrix

scikit-learn · Samples, features and targets

https://scikit-learn.org/stable/getting_started.html

pandas · Time-based joins

https://pandas.pydata.org/docs/reference/api/pandas.merge_asof.html

scikit-learn · Cross-validation

https://scikit-learn.org/stable/modules/cross_validation.html

Quellen · tsfresh

tsfresh · Feature extraction

https://tsfresh.readthedocs.io/en/latest/text/introduction.html

tsfresh · Data formats

https://tsfresh.readthedocs.io/en/latest/text/data_formats.html

tsfresh · Extraction settings

https://tsfresh.readthedocs.io/en/latest/text/feature_extraction_settings.html

tsfresh · Quick start

https://tsfresh.readthedocs.io/en/latest/text/quick_start.html

Quellen · DTW und Transformation

tslearn · Dynamic Time Warping

https://tslearn.readthedocs.io/en/stable/user_guide/dtw.html

tslearn · dtw_path

https://tslearn.readthedocs.io/en/stable/gen_modules/metrics/tslearn.metrics.dtw_path.html

scikit-learn · Leakage and preprocessing

https://scikit-learn.org/stable/common_pitfalls.html